

- **Consistent hashing:** Memcached decides the node to store/retrieve the data on, when it receives a request, irrespective of the total number of Memcached nodes active. Rather than choosing a random node, or using the round-robin method to load-balance all the nodes, it hashes all the incoming keys, which decides the node on which the data is going to be stored. This hash function evenly distributes the keys among all the nodes. The algorithm used for hashing is complex enough to compensate for a node failure, so that the calculation does not change when the total number of nodes changes, and there is no ambiguity while retrieving data. The application basically has to be aware about all the nodes that are in the Memcached cluster, and you need to populate it with a list of all the IPs and ports on which the Memcached servers are active. To know more about this algorithm, check out reference [4] in the *Further reading* section.
- **LRU algorithm:** Memcached uses a predefined portion of maximum memory, so if you ask it to use a maximum of 1 GB RAM, it will try not to use memory beyond that. But what happens if a request comes in to store data, and you don't have memory left? Memcached uses the LRU (Least Recently Used) algorithm to determine which data in memory has been least active, and discards it to make space for the new data. This is where the main concept behind caching lies, i.e., you cannot rely on Memcached to keep some data once it has been asked to store it into memory. It all depends on the amount of traffic that the particular key has received, and it is really a very efficient way of managing memory. So while developing your application, remember that you must have a fallback mechanism to retrieve the data once it is not found in the Memcached server.
- **Memory allocation:** Memcached is written in the C programming language, so it has to manage its own memory allocation schemes. Many think it uses the *malloc* function to allocate free memory from the RAM—but that results in some major address-space fragmentation issues once you start allocating large chunks of memory. It also results in performance degradation once the application has been running for a fair amount of time and there have been many allocate-reallocate cycles. To avoid such a scenario, Memcached uses a slab memory allocator. In order to optimise the cache usage of your application, you need to know how this slab allocation works. Basically, it's like any other memory management scheme, where the memory is divided into a fair number of pages. Each page is assigned a particular slab class, according to which the page is divided into a finite number of equal-sized chunks. The chunk

IMPORTANT FUNCTIONS

A Memcached client mostly has the following functions available, whatever the programming environment it may be in:

- *get(key)*—Retrieve the value associated with the specified key
- *set(key, value, expiry)*—Add or replace the given key's value along with the given expiry times
- *add(key, value, expiry)*—Add a value associated with the new key in Memcached; if the key already exists, it returns an error.
- *append(key)*—Add the current data at the end of the value that is already associated with the key.
- *prepend(key)*—Add the current data before what already exists, and which is associated with the key.
- *delete(key)*—Delete or invalidate the key.
- *replace(key, value)*—Replace the value of the existing key in Memcached. Returns an error if the associated key does not already exist.
- *flush_all()*—Invalidates all keys in Memcached memory.

size depends on the slab class that the page has been assigned. The page a particular data is assigned to depends on the actual size of the data, including the key and the value, according to which it is given a page of a relevant slab class. For example, a page that is 1 MB in size and belongs to Slab Class 1 will have eight 128 KB sized chunks. So, if a key-value pair is 109 KB in size, it is likely to be assigned a chunk of 128 KB in a page of this slab class (yes, 19 KB is wasted, but that's the trade-off). Now, it is important to note that the LRU algorithm discussed earlier works only on a per-slab basis. So, a key is likely to be evicted earlier if the slab it belongs to is being used more than others.

Installing Memcached

While installing Memcached, we need to install the daemon and the client library. Also, Memcached has a dependency on *libevent*, which you need to install before Memcached. But, on most major Linux distributions, it is almost as easy as *sudo apt-get install memcached* or *yum install memcached*. Alternatively, you might want to compile from source, if you want a more recent version. In that case, you can get the *libevent* dependency at <http://libevent.org> and Memcached itself at <http://memcached.org>. You will, of course, also need all the build dependencies.

Since all Memcached servers in the cluster do not need to know about each other, there is almost no other configuration that you need to do in order to get up and running. To start the Memcached instance, run the following:

```
memcached -d -m 1024 -u memcache -l 127.0.0.1 -p 11211
```